AI Interview Copilot: Case Study

Problem

AI Interview Copilot is a web app for job seekers. During a live interview on Zoom, Google Meet or Teams, it listens to the meeting tab, transcribes what the interviewer asks and shows a suggested answer on screen, grounded in the candidate's own CV and the job description.

A live interview leaves no time to plan a structured answer. General chat tools do not help: the candidate has to type the question, wait, and then check whether the answer claims experience they do not have. A suggestion that arrives late is useless, and one that invents a skill or a number is a risk in front of the interviewer.

The product needed to hear the question without any typing, answer while the question is still fresh, stay within what the CV actually says, and run on free infrastructure with no payment card. I built a real-time pipeline that takes browser audio and streams back a CV-grounded answer in under two seconds.

Metric	Before	After
Wait until answer text starts to appear (15 questions)	2.0 s median (plain chat prompt, full answer)	0.6 s median (copilot, streamed)
Answers with figures not in the CV or job	12 of 15	0 of 15
Answers short enough to say aloud	0 of 15 (median 380 words)	15 of 15 (median 100 words)
Answers claiming a job skill not on the CV	2 of 12	0 of 12

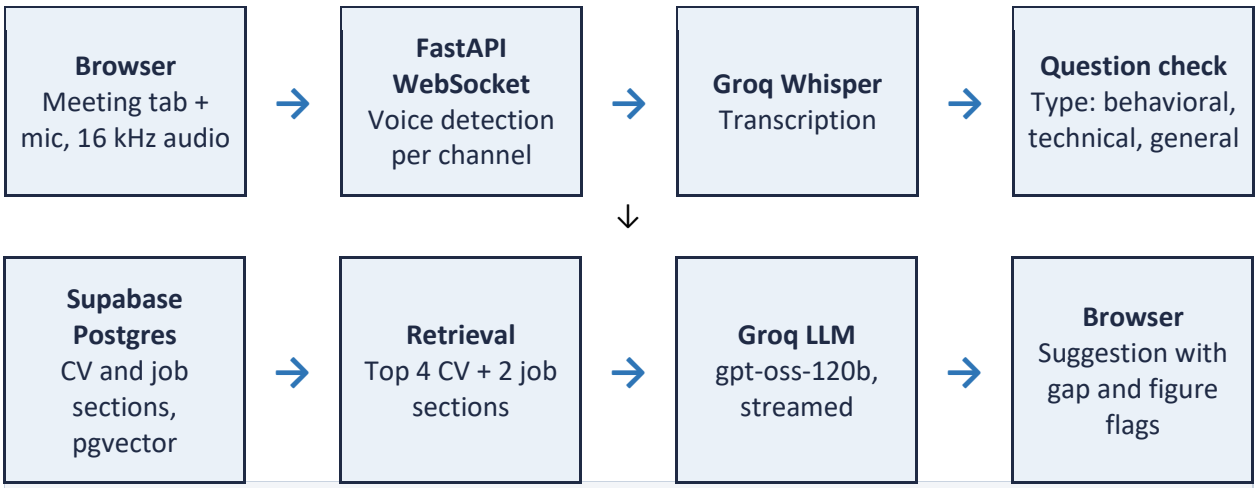
Challenges

Challenge	How It Was Solved
Knowing when the interviewer has finished a question	An energy-based voice detector ends an utterance after 0.8 s of silence and a question check decides when to answer. A follow-up asked within 6 s joins the same question.
Answers inventing experience	The prompt uses only the 4 most relevant CV sections and 2 job sections. Job skills the CV does not show are listed once per session, banned from answers and flagged if used, and any figure not in the CV is highlighted.
Speed on free-tier models	Groq Whisper for speech and gpt-oss-120b at low reasoning effort for answers, streamed word by word. Question embedding runs locally on CPU in about 16 ms.

Challenge	How It Was Solved
Vercel closes WebSockets after 300 s	The server announces the limit. The browser opens a second connection at a quiet moment, moves the audio to it and lets the old one finish its answer, flushing any cut sentence.
Backend over Vercel's 225 MB function limit (229 MB)	I replaced the fastembed library with onnxruntime and a plain-Python tokenizer that give identical vectors, bringing the dependencies to about 179 MB by the same count.

Architecture

Audio moves from the browser to the backend over a WebSocket, through transcription and question detection, and comes back as a streamed answer.



Interviewer and candidate audio are separate channels, and every transcript line and answer is also saved to Postgres as interview history.

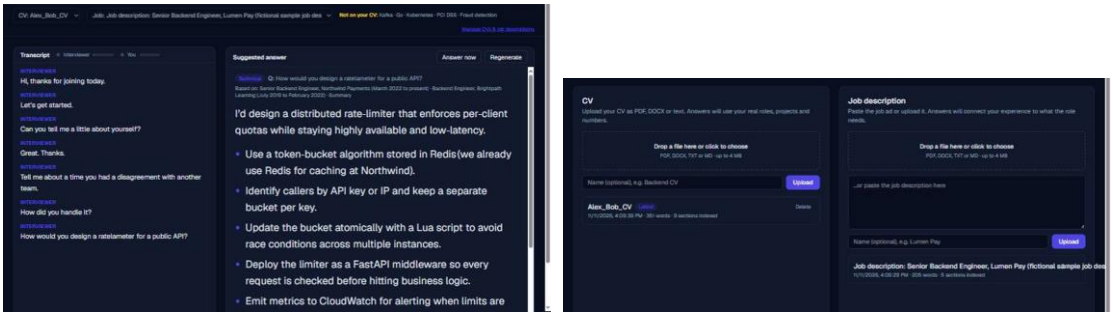
What I Built

Component	What It Does
Realtime session	Takes the meeting tab and microphone as two 16 kHz channels, transcribes utterances in order and starts an answer when the interviewer asks a question.
Answer engine	Classifies each question and builds a prompt from the most relevant CV and job sections plus the recent conversation.
Document pipeline	Parses PDF, DOCX and text uploads into sections that keep role, company and dates, embeds them locally with bge-small-en-v1.5 on onnxruntime and stores them in pgvector.
Honesty checks	Lists job skills missing from the CV and flags any unsupported figure or missing skill that an answer mentions.

Component	What It Does
Connection switcher	Keeps a session running past Vercel's 300 s WebSocket limit by switching connections at a quiet moment.
Accounts and history	Supabase sign-in checked on every API and WebSocket call, with each interview's transcript and answers saved.
Evaluation and latency tracking	Scripts replay a mock interview and score a labelled question set, and an admin page shows median, p90 and p95 answer timings without storing content.
Deployment and CI	Website and API run as two Vercel projects in Tokyo next to the database, and every push runs the test suite.

How It Works

Step	What Happens
1	The candidate uploads a CV and pastes or uploads the job description. Each is split into sections, embedded and stored.
2	On the copilot page they choose the CV and job, press Start, share the meeting tab with audio and allow the microphone. The WebSocket signs in with their access token.
3	Audio streams in small chunks. The backend closes an utterance after 0.8 s of silence and Groq Whisper transcribes it, with a partial transcript every 2 s while the interviewer talks.
4	When the interviewer's sentence is a question, the backend classifies it, retrieves the 4 best CV sections and 2 job sections, and streams an answer from the LLM.
5	The answer appears word by word with its type, the CV roles it drew on, and highlights for any figure or job skill the CV does not support.
6	Transcript and answers are saved to the interview history. Before Vercel's 300 s cut, the browser moves to a fresh connection without interrupting the audio.



Left: a live session on the deployed site, with the interviewer's questions transcribed and a technical answer drawn from Alex Bob's CV. Right: the CV and job description split into indexed sections after upload.

Key Code

Question detection and follow-up merge (backend/app/realtime/session.py)

```
def _on_interviewer_final(self, text: str, speech_end: float, stt_ms: int) -> None:
    self._last_interviewer_text = text
    self._pending_question.append(text)
    if not is_question(text):
        return
    question = " ".join(self._pending_question)
    self._pending_question.clear()
    now = time.monotonic()
    if (
        self._last_question
        and self._last_auto_question_at is not None
        and now - self._last_auto_question_at < FOLLOW_UP_WINDOW_S
        and not self._candidate_spoke
    ):
        question = f"{self._last_question} {question}"
    self._last_auto_question_at = now
    self._candidate_spoke = False
    self._start_answer(question, timings={"stt_ms": stt_ms, "_speech_end": speech_end})
```

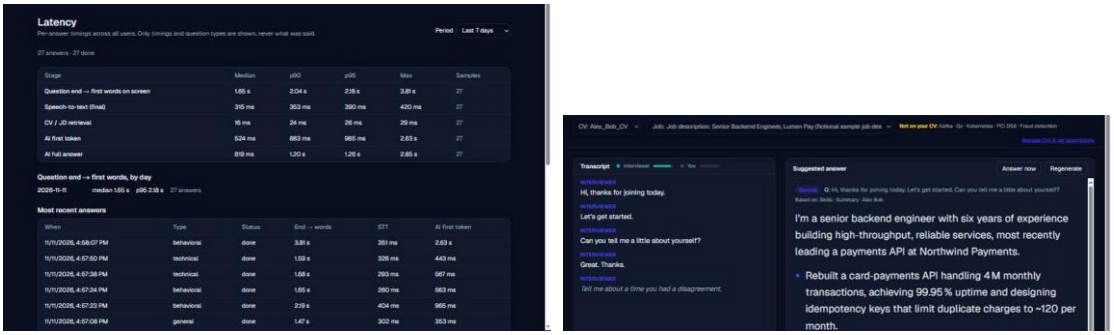
Switching connections at a quiet moment (frontend/src/lib/copilotSocket.ts)

```
private promoteWhenQuiet(next: Conn) {
  const old = this.current;
  if (this.switching !== next || this.closedByUser || next.ws.readyState !==
WebSocket.OPEN) {
    return;
  }
  const forced = !old || Date.now() >= this.deadline(old, forceBefore(old));
  if (old && old.speaking.size > 0 && !forced) {
    next.timers.push(setTimeout(() => this.promoteWhenQuiet(next), POLL_MS));
    return;
  }
  this.switching = null;
  this.current = next;
  this.attempt = 0;
  this.handlers.onStatus("open");
  if (old) {
    // Forced mid-sentence: have the old connection finish what it heard.
    if (old.speaking.size > 0) this.sendOn(old, { type: "flush" });
    this.retire(old);
  }
}
```

Results

	Result
✓	27 live answers on the deployed site: first words on screen a median 1.65 s after the question ended (p95 2.18 s), with speech-to-text at 315 ms and retrieval at 16 ms.

	Result
✓	Before/after test on the same 15 questions, CV, job and model: the plain chat prompt put figures not in the CV into 12 answers and the copilot into 0; 0 plain answers were short enough to say aloud against 15 from the copilot.
✓	Labelled set of 15 questions: 15 of 15 given the right format, 13 of 15 with no unsupported figures, median 593 ms to the first word from the model (1,284 ms max).
✓	Connection switching tested live: an 85-second interview with a 45-second limit switched twice with no lost words.
✓	Deployment fixes verified: the new embedder matches fastembed's vectors exactly and fits Vercel's 225 MB limit, and six instances starting together went from 4 deadlocks to 0.
✓	181 automated tests pass. The deployed site was checked end to end: sign-in, dashboard, document indexing on Vercel and a WebSocket connection in 0.8 s.



Left: the admin latency page on the deployed site, 27 answers with question end to first words at a 1.65 s median. Right: session start with the Lumen Pay job, with Kafka, Go, Kubernetes, PCI DSS and fraud detection flagged as not on the CV.

Conclusion

The copilot works end to end on free infrastructure: it hears the question, starts answering about 1.7 seconds after the interviewer stops, and stays within what the CV supports. The accuracy figures come from one sample CV and job pair and a recorded mock interview, so they are signals rather than guarantees. The backend also ships as a Docker image, so it can move to an always-on server when sessions need to run without Vercel's connection limit.